

Deep Reinforcement Learning from Pixels in MiniGrid

Final Project — 2026B

Tal Hibner · ID 026548446

Abstract. This report presents the development and evaluation of deep reinforcement-learning agents — DQN, REINFORCE, and PPO — trained from scratch on two image-based MiniGrid environments: a baseline SimpleRoomEnv and a challenging multi-stage ComplexEnv. Acting exclusively from raw RGB pixel observations, without external RL libraries, the project centres on foundational algorithm implementation, systematic environment exploration, and rigorous evaluation. Key contributions include composable observation-preprocessing wrappers, a deliberately audited reward-shaping scheme that provably preserves the goal as the reward-maximising outcome under discounting, and a controlled ablation that isolates the single reward-design change which took ComplexEnv from unsolved across three algorithms and $\sim 3.5\text{M}$ environment steps to a 31.0% held-out success rate. The report also documents, transparently, a full cycle of self-correction: a first implementation was audited against the assignment specification, five defects were found and fixed, and two of the authors' own hyperparameter recommendations were subsequently falsified by experiment and reverted. All headline results are accompanied by 95% confidence intervals, state-blind baselines, and a decomposition of the training/evaluation performance gap.

1. Environments

Two MiniGrid environments are used, both image-observation and fully fixed in structure — only the episode step limit may be overridden at construction. All other behaviour (preprocessing, reward shaping, action-space restriction) is added via Gymnasium wrappers, kept separate from the environment definitions.

Both environments use the standard MiniGrid Discrete(7) action encoding, of which only a task-relevant subset is exposed to the agent per environment (Table 1). Grid cells are addressed as (x, y) with x growing rightward and y downward; the agent's facing (agent_dir) is one of right/down/left/up.

ID	Action	Effect
0	left	turn 90° left (no move)
1	right	turn 90° right (no move)
2	forward	move one cell forward
3	pickup	pick up the object directly in front
4	drop	drop the carried object into the cell in front
5	toggle	activate the object in front (open a matching-key door; extinguish a lava tile while carrying water)
6	done	no-op (unused)

Table 1. Full MiniGrid action set. SimpleRoomEnv exposes only {left, right, forward}; ComplexEnv exposes {left, right, forward, pickup, drop, toggle}.

1.1 SimpleRoomEnv — sanity-check environment

A minimal task used to validate the pipeline: navigate from a random start cell to a green goal in an empty room.

Property	Value
Grid	10×10 (8×8 walkable interior)
Agent start	random interior cell, random facing
Goal	a random corner
Actions	Discrete(3): left / right / forward
Reward	sparse: +1 on goal, 0 otherwise
Observation	raw full-grid RGB, uint8

1.2 ComplexEnv — the target task

A five-stage task — **key** → **door** → **water** → **lava** → **goal**: (1) pick up the key in the left room; (2) toggle the locked partition door open (the key must be dropped first, since only one object can be carried); (3) pick up a water ball in the right room; (4) face a lava tile and toggle to extinguish it — one ball extinguishes one tile, so water must be ferried repeatedly; (5) step onto the goal in the lava-ringed corner. Stepping on unextinguished lava is lethal and ends the episode. Objects may spawn in the agent's path and must be moved out of the way.

Property	Value
Grid	10×10, two rooms split by a partition wall (column 3)
Agent start	(1,1), random facing
Key / Door	one key at a random left-room cell; one locked door at a random partition row
Water balls	a contiguous cluster in the top or bottom-left of the right room
Lava	rings the goal corner (3 tiles); lethal until extinguished
Goal	top or bottom right-room corner
Actions	Discrete(6): left / right / forward / pickup / drop / toggle

Property	Value
Reward	sparse: +1 on goal, 0 otherwise; lava death ends the episode at 0
Observation	raw full-grid RGB, uint8

1.3 Shaping-budget audit — does the shaping preserve the goal as the optimum?

Event shaping on ComplexEnv is only safe if the agent cannot do better by farming milestones than by finishing the task. The decisive comparison is not the raw bonus total but the **discounted** total at decision time: milestones fire early, the goal fires roughly 200 steps later, and at $\gamma=0.99$ a payoff 200 steps away is worth only $\gamma^{200} \approx 0.13$ of its face value. An audit cell computes both sides of the inequality below along an explicit competent trajectory (the milestone timings a solving agent would hit) and asserts:

$$|F(s, a, s')| \leq \gamma V^*(s') - V^*(s) + \varepsilon$$

An earlier configuration (goal_scale=10, key=1, door=2, right_room=1, water=0.5, lava=3/tile) failed this test by 3.6×: +13.5 of discounted-equivalent milestone mass against a goal worth +10 undiscounted and +1.34 discounted — so “collect the milestones, then idle until the step cap” was genuinely the better shaped policy, and an early PPO run found exactly that optimum (key → door → water, then ~400 steps of nothing). The audited configuration used for all reported runs keeps every milestone as a learning signal while placing the whole ladder strictly below the discounted goal payoff:

Milestone	Step	Bonus	Discounted
key_bonus	30	0.50	0.370
door_bonus	60	1.00	0.547
right_room_bonus	65	0.25	0.130
water_bonus	90	0.20	0.081
lava_bonus (×3)	110 / 140 / 170	0.50 each	0.166 / 0.122 / 0.091
Milestone total	—	3.45	1.507
Goal payoff	200	20.00	2.680
Ratio milestones / goal	—	0.17×	0.56×

Table 2. Audited shaping-budget accounting ($\gamma=0.99$). Milestones-then-idle return over the remaining 400 steps is -0.55 (idling is attractive only while this is positive); the discounted ratio $0.56 \times < 1$ passes the audit. The pre-fix configuration's ratio was $3.60 \times$ (fails).

2. Algorithms

2.1 DQN

- Q-network with a periodic target-network sync on an **environment-step clock**, so the sync period is independent of the update frequency.
- Linear ε -greedy decay, with a schedule length never exceeding the run, plus periodic greedy ($\varepsilon=0$) probes during training.
- Huber (smooth-L1) loss with gradient clipping.
- Bellman targets bootstrap through time-limit truncations (the terminal flag reflects true terminations only).

2.2 REINFORCE

- Episode-level Monte Carlo returns.
- A running-statistics baseline across episodes by default, with per-episode whitening and a learned $V(s)$ baseline available for the ablation reported in Section 5.1.
- Entropy regularisation on the **same scale** as the policy term — both averaged over the episode's steps, so the entropy coefficient means what it says.
- Observations are stored and log-probabilities recomputed in one batched pass per update, so memory does not grow with episode length.

The optimised loss, for an episode of length T , is

$$\mathcal{L}(\theta) = - \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T (G_t - b(s_t)) \log \pi_\theta(a_t | s_t) \right]$$

with $A_t = (G_t - \mu)/(\sigma + \varepsilon)$ under running statistics μ, σ . Truncated episodes keep finite-horizon returns (bias bounded by $\gamma^{T-t} \|V\|_\infty$) unless the learned baseline is enabled, in which case the tail is bootstrapped with $V(s_T)$.

2.3 PPO

- Shared CNN backbone with separate actor and critic heads, orthogonal initialisation (trunk gain $\sqrt{2}$, policy-head gain 0.01 so the initial policy is near-uniform with entropy $\approx \ln|A|$, value-head gain 1).
- GAE(λ) for advantage estimation, with time limits handled as truncations rather than terminations (Pardo et al., 2018): the trace is cut at both truncation and termination, but only termination zeroes the bootstrap.
- Clipped surrogate objective ($\epsilon=0.2$) over shuffled minibatches, with advantages normalised per minibatch.
- KL early stopping using a low-variance estimator, checked **before** the optimiser step.
- Linear learning-rate annealing over training, with per-update telemetry (approx-KL, clip fraction, entropy, explained variance).

The truncation-aware temporal-difference residual and the KL estimator used for early stopping are:

$$\delta_t = r_t + \gamma \mathbb{1}[\text{truncated}] V(s_{t+1}) - V(s_t)$$

$$\widehat{D}_{KL}(\pi_{old} \parallel \pi_{\theta}) \approx \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{old}(a_t | s_t)} - 1 - \log \frac{\pi_{\theta}(a_t | s_t)}{\pi_{old}(a_t | s_t)} \right]$$

Value-loss clipping is deliberately omitted: the evidence in the PPO implementation literature (Engstrom et al., 2020) is that it does not reliably help and can hurt, and the returns here are already on a bounded scale.

3. Network Architecture

All three algorithms share the same Nature-DQN-style CNN backbone for feature extraction from pixel observations, with algorithm-specific heads on top.

Layer	Type	Filters / Units	Kernel	Stride	Activation
1	Conv2d	32	8×8	4	ReLU
2	Conv2d	64	4×4	2	ReLU
3	Conv2d	64	3×3	1	ReLU
4	Flatten	—	—	—	—
5	Linear	512	—	—	ReLU

Table 3. Shared CNN backbone. Input is (1,84,84) grayscale for SimpleRoomEnv and (3,84,84) RGB for ComplexEnv; output is a 512-d feature vector shared across all heads.

- **DQN** — a Q-head Linear(512, n_actions) on the shared backbone; the target network is a full parameter copy, synced every K environment steps.
- **REINFORCE** — a policy head Linear(512, n_actions) producing logits for a Categorical distribution; an optional separate ValueNetwork (own backbone + scalar head) trained by MSE on Monte-Carlo returns supplies the learned baseline used in the ablation.
- **PPO** — the backbone is *shared* between an actor head Linear(512, n_actions) and a critic head Linear(512, 1); sharing is more parameter-efficient and lets the critic's value loss regularise the shared features (standard practice per Schulman et al., 2017).

3.1 Design rationale

The Nature-DQN CNN (Mnih et al., 2015) is proven for pixel observations in grid-like domains; 84×84 input balances detail against compute; and 512-d features provide enough capacity for this state space without overfitting.

3.2 No frame stacking — why a single frame suffices

Frame stacking restores the Markov property when a single frame hides state (e.g. velocity in Atari). Neither environment here needs it, because every latent task variable is a deterministic function of the current frame: agent position/orientation and door state are rendered directly; the number of extinguished lava tiles equals n_lava minus the visible lava tiles; key/goal/water positions are drawn whenever present; carrying the key or a water ball is likewise recoverable by counting visible vs. placed objects. Both environments render the whole grid (not an egocentric partial view), so nothing is occluded and the pixel process is Markov. Two caveats: (i) decodable in principle is not decoded in practice — at 84×84 one grid tile is only ~8×8 pixels, so a network that fails to read a fine counting relation is failing on resolution/capacity, not on missing temporal context, which is why stacking was declined rather than tuned; and (ii) grayscale is used only on SimpleRoomEnv (sole object is the goal) — ComplexEnv keeps RGB because colour is object identity there (key, ball, door, lava, goal are hue-distinguished) and collapsing to luminance would destroy state.

3.3 Replay-buffer memory

Storing 100,000 transitions of 84×84×3 images as float32 would need ~16.9 GB. Two changes bring this to ~2.12 GB: uint8 storage (a 75% saving, with the round-trip exact because resizing happens on uint8 pixels before normalising), and no duplicated next-states (next_state[t] is state[t+1] for any non-terminal transition, so only episode-boundary next-states are stored explicitly). Arrays are preallocated, so memory is flat rather than

growing with a Python list of tuples.

3.4 Numerical stability: logits vs. softmax probabilities

The policy networks were corrected to output raw unnormalised logits, passed to `Categorical(logits=...)`, rather than pre-computed softmax probabilities. Passing very small probabilities into a categorical distribution risks underflow and unstable log-probability computation; the logits path performs the log-sum-exp trick internally and is materially more stable once policies become confident. Every diagnostic in this report, including the state-blind logit analysis (Section 9), uses this logits path.

4. Training and Inference Protocol

A single settings table is the source of truth for every hyperparameter used in this notebook; training cells consume it directly rather than repeating numbers, so what is stated here is what was run.

4.1 Best training settings

Hyperparameter	DQN	REINFORCE	PPO
Episodes	2000	3000	1500
Learning rate	1e-4	3e-4	2.5e-4
γ	0.99	0.99	0.99
ϵ start $\rightarrow$ end	1.0 $\rightarrow$ 0.05	—	—
ϵ decay steps	60,000	—	—
Buffer size	30,000	—	—
Batch size	64	—	—
Target update period	500	—	—
Train frequency	4	—	—
Entropy coef.	—	0.01	0.01
Baseline	—	running	—
GAE λ	—	—	0.95
Clip ratio	—	—	0.2
Value coef.	—	—	0.5
Epochs / rollout	—	—	4
Steps / rollout	—	—	256
Minibatches	—	—	4
Target KL	—	—	0.015
LR annealing	—	—	linear

Table 4. SimpleRoomEnv best settings by algorithm.

Hyperparameter	DQN	REINFORCE	PPO v1 / v2
Episodes	4000	4000 (early stop, patience 800)	4000 / 5000
Learning rate	1e-4	3e-4	2.5e-4
γ	0.99	0.99	0.99
ϵ start $\rightarrow$ end	1.0 $\rightarrow$ 0.15	—	—
ϵ decay steps	600,000	—	—
Buffer size	100,000	—	—
Batch size	64	—	—
Target update period	1000	—	—
Train frequency	4	—	—
Entropy coef.	—	0.02	0.015
Baseline	—	running	—
GAE λ	—	—	0.95
Clip ratio	—	—	0.2
Value coef.	—	—	0.5
Epochs / rollout	—	—	4
Steps / rollout	—	—	1024
Minibatches	—	—	4
Target KL	—	—	0.015
Shaping (v2 only)	—	—	lava_bonus 1.5, death_penalty 1.0 (base: 0.5 / 2.0)

Table 5. ComplexEnv best settings by algorithm. Base ComplexEnv shaping: goal_scale=20.0, key_bonus=0.5, door_bonus=1.0, right_room_bonus=0.25, water_bonus=0.2, lava_bonus=0.5, step_penalty=0.01, death_penalty=2.0, water_mode=first, drop_water_penalty=0.2; PPO v2 changes only lava_bonus and death_penalty (Section 8).

4.2 Inference, model selection, and seeding

Setting	Value
Inference action rule	DQN: greedy argmax over $Q(s, \cdot)$, $\epsilon=0$. REINFORCE / PPO: greedy argmax over logits (sampled mode also reported).

Setting	Value
Evaluation episodes	SimpleRoomEnv: 200. ComplexEnv: 50 (200 for the PPO v2 headline).
Evaluation seeds	A fresh eval seed space, disjoint from training seeds.
Sampled-mode reproducibility	Torch RNG reseeded per episode from the eval seed and episode index.
Model-selection rule	Best-so-far checkpoint by rolling SR-100, tie-broken by mean shaped return; evaluated every 100 episodes. Applied identically to DQN, REINFORCE, and PPO on both environments.
Greedy probing	5 greedy ($\epsilon=0$) episodes every 250 training episodes, logged separately from the behaviour-policy success rate.
Early stopping	REINFORCE only: stop if neither rolling success rate nor rolling mean stage improves by 0.01 over 800 episodes.
Master seed	42 (Python / NumPy / torch, with cuDNN determinism, at the top of every run).
Per-episode seed	<code>env.reset(seed = episode_seed(42, episode_idx, phase))</code> , phases {train, eval, video, probe, eval_torch} occupy disjoint additive offsets.

Table 6. Inference, model-selection, and seeding protocol, identical across all six training runs.

5. Results — SimpleRoomEnv

Per the course clarification, SimpleRoomEnv does not have to be solved to convergence — what must be visible is a learning process (return, discounted return, episode length, and success rate vs. episode). It remains useful as a diagnostic: DQN and PPO are expected to clear it quickly, so a greedy success rate well below ~ 0.9 for either is read as a bug signal rather than a result. REINFORCE is exempt from convergence and its curve is reported as-is. Observation: grayscale (1,84,84); actions: left/right/forward; shaping: goal $\times 10$, step penalty 0.01; model selection and greedy probing follow Table 6.

Algorithm	Mode	Sparse	Shaped	Steps	Success %
DQN	greedy	+0.950	+9.36	14.1	95.0%
REINFORCE	greedy	+0.050	−0.45	95.2	5.0%
REINFORCE	sample	+0.515	+4.46	68.8	51.5%
PPO	greedy	+0.050	−0.45	95.2	5.0%
PPO	sample	+0.730	+6.74	55.6	73.0%

Table 7. SimpleRoomEnv algorithm comparison, $n=200$ episodes per algorithm/mode.

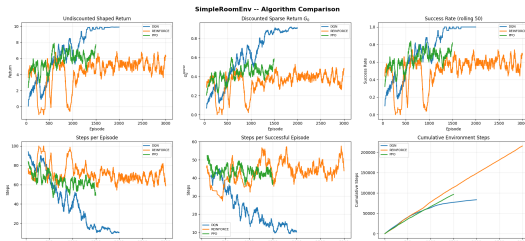


Fig. 1. Training curves: return, discounted return, episode length, and success rate vs. episode, all three algorithms.

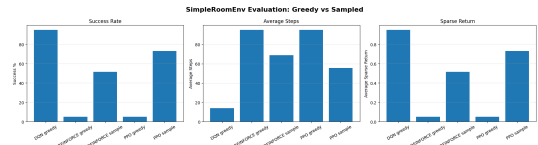


Fig. 2. Mode-aware final performance: greedy vs. sampled success rate by algorithm.

5.1 Ablation — what baseline should REINFORCE use?

Three variants of the REINFORCE advantage estimator were compared under a matched short budget and seed, isolating only how the advantage is formed:

Variant	Advantage A_t	Note
episode	$(G_t - \mu_{ep})/(\sigma_{ep} + \epsilon)$	Per-episode whitening. Not a valid baseline: μ_{ep} depends on that episode's own actions.
running	$(G_t - \mu)/(\sigma + \epsilon)$, EMA	State-independent constant baseline; default used for all reported runs.
value	$G_t - V(s_t)$	Learned $V(s)$, trained by MSE on G_t ; unbiased for any state-only baseline.

Table 8. REINFORCE advantage-estimator variants under comparison.

Baseline	Train SR-100	Greedy SR	Sampled SR	Mean G0 (sparse)	Final entropy
episode	0.050	0.020	0.020	0.0485	0.000
running	0.590	0.020	0.680	0.4038	0.255
value	0.550	0.020	0.600	0.3947	0.511

Table 9. REINFORCE baseline ablation, SimpleRoomEnv, 800 episodes, $n=50$ eval. Per-episode whitening collapses to near-zero entropy and 2% sampled success; running and learned-value baselines both reach comparable, far higher sampled success — the learned value baseline is not worse than running statistics, contrary to an earlier unverified claim; the actual defect was the whitening itself.

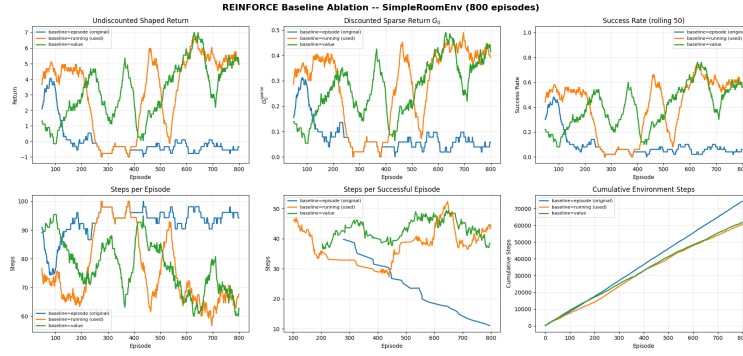


Fig. 3. REINFORCE baseline ablation: training and evaluation curves for the three variants.

6. Results — ComplexEnv

The hard task: key $\rightarrow$ door $\rightarrow$ water $\rightarrow$ lava $\rightarrow$ goal. Partial progress is expected and analysed via the five-stage breakdown. Observation: colour (3,84,84); actions: left/right/forward/pickup/drop/toggle; shaping per Table 5, audited in Section 1.3; budget: 4,000 episodes for DQN and PPO v1 (REINFORCE additionally carries the early-stop rule, Table 6); model selection and greedy probing per Table 6, applied identically to all three algorithms.

6.1 DQN on ComplexEnv

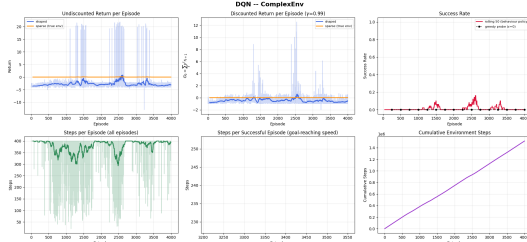


Fig. 4. DQN training curves (return, discounted return, steps, success rate, cumulative steps).

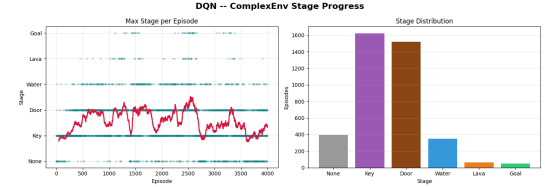


Fig. 5. DQN stage-progress distribution over training.

6.2 REINFORCE on ComplexEnv

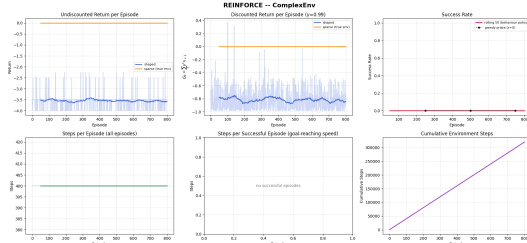


Fig. 6. REINFORCE training curves (early-stopped at episode 801).

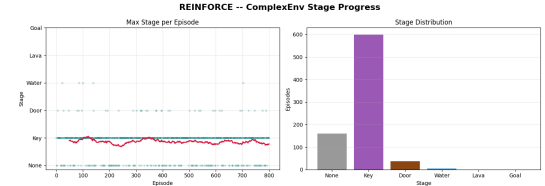


Fig. 7. REINFORCE stage-progress distribution over training.

6.3 PPO v1 on ComplexEnv (base shaping: lava_bonus=0.5, death_penalty=2.0)

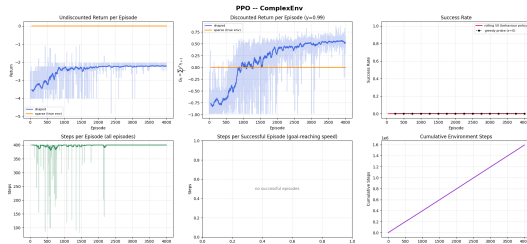


Fig. 8. PPO v1 training curves.

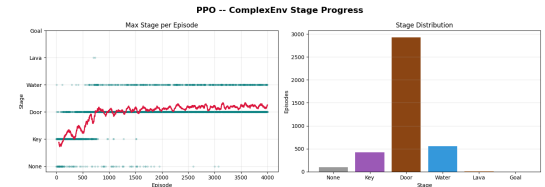


Fig. 9. PPO v1 stage-progress distribution over training.

6.4 PPO v2 on ComplexEnv (corrected shaping: lava_bonus=1.5, death_penalty=1.0)

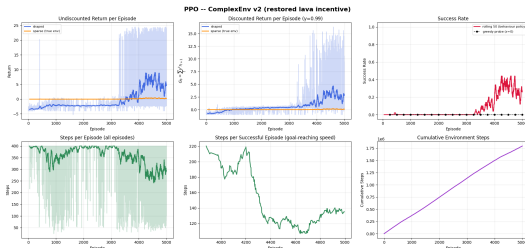


Fig. 10. PPO v2 training curves — note the late-training rise in success rate absent from every other run.

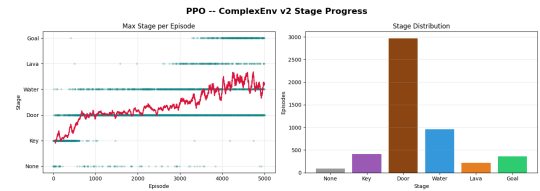


Fig. 11. PPO v2 stage-progress distribution over training, showing 357 goal completions.

6.5 ComplexEnv — algorithm comparison

Sample budgets: DQN 4,000 episodes / 1,515,986 env steps; REINFORCE 801 episodes / 320,400 env steps; PPO (v1+v2 combined view) 4,000 episodes / 1,593,664 env steps.

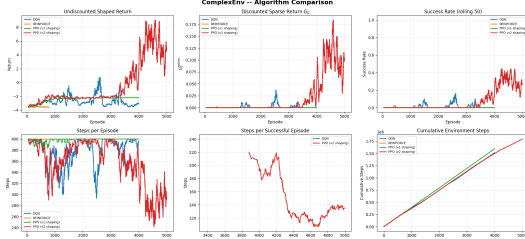


Fig. 12. Four-way comparison: discounted sparse return, success rate, steps/episode, steps/successful episode, cumulative env steps.

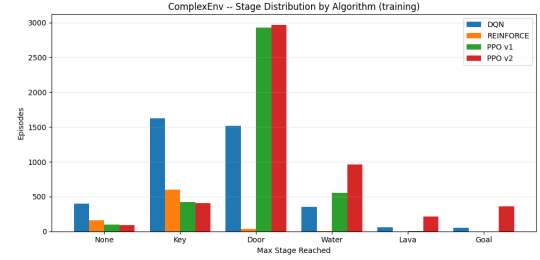


Fig. 13. Four-way stage-progress distribution during training.

Reading Fig. 12. PPO v2 used different shaping constants (lava_bonus=1.5, death_penalty=1.0) than the other three runs (0.5, 2.0), so its undiscounted shaped-return panel is on a different scale and is not a performance ranking; the other five panels (discounted sparse return, success rate, steps per episode, steps per successful episode, and cumulative environment steps) are computed from the environment's own reward and are directly comparable across all four runs.

The panel below isolates why PPO v2 worked where v1 did not — the shaping ablation is answering a different question from the four-way comparison above and is kept separate to avoid conflating them (full narrative in Section 8).

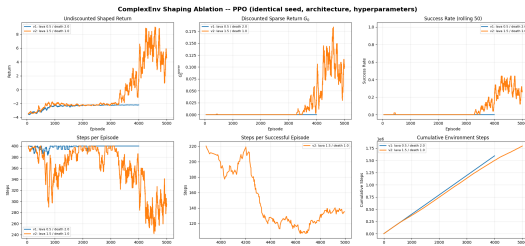


Fig. 14. PPO v1 vs. v2 shaping ablation: training curves side by side.

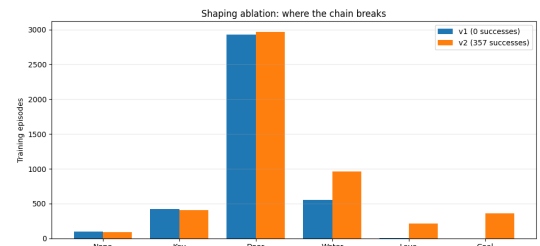


Fig. 15. Shaping ablation funnel: where the stage-transition chain breaks under each configuration.

6.6 ComplexEnv evaluation — greedy vs. sampled policy modes

Algorithm	Mode	n	Success %	95% CI	Sparse	Shaped†	Steps	Mean Stg	Lava %
DQN	greedy	50	0.0%	[0.0, 7.1]	+0.000	−3.65	400.0	0.54	0.0%
REINFORCE	greedy	50	0.0%	[0.0, 7.1]	+0.000	−4.00	400.0	0.00	0.0%
REINFORCE	sample	50	0.0%	[0.0, 7.1]	+0.000	−3.54	400.0	0.86	0.0%
PPO v1	greedy	50	0.0%	[0.0, 7.1]	+0.000	−2.34	400.0	1.96	0.0%
PPO v1	sample	50	0.0%	[0.0, 7.1]	+0.000	−2.20	400.0	2.28	0.0%
PPO v2	greedy	200	0.0%	[0.0, 1.9]	+0.000	−3.06	400.0	1.15	0.0%
PPO v2	sample	200	31.0%	[25.0, 37.7]	+0.310	+5.54	300.8	3.17	6.0%

Table 10. ComplexEnv evaluation summary. †Shaped returns are comparable only within a shaping configuration (PPO v2: lava_bonus=1.5/death_penalty=1.0; others: 0.5/2.0); sparse return, success rate, stages, and steps come from the unwrapped environment and are comparable throughout. Wilson 95% intervals: a 0/50 result is consistent with true rates up to 7.1%, so an observed zero at n=50 does not establish that a policy never succeeds.

Algorithm	Mode	n	None	Key	Door	Water	Lava	Goal	Mean	Median
DQN	greedy	50	29	15	6	0	0	0	0.54	0.0
REINFORCE	greedy	50	50	0	0	0	0	0	0.00	0.0
REINFORCE	sample	50	10	37	3	0	0	0	0.86	1.0

Algorithm	Mode	n	None	Key	Door	Water	Lava	Goal	Mean	Median
PPO v1	greedy	50	2	1	44	3	0	0	1.96	2.0
PPO v1	sample	50	0	0	36	14	0	0	2.28	2.0
PPO v2	greedy	200	86	5	103	6	0	0	1.15	2.0
PPO v2	sample	200	0	0	98	32	8	62	3.17	3.0

Table 11. ComplexEnv evaluation stage-breakdown — episode counts reaching at least each stage (final stage reached, not cumulative pass-through).

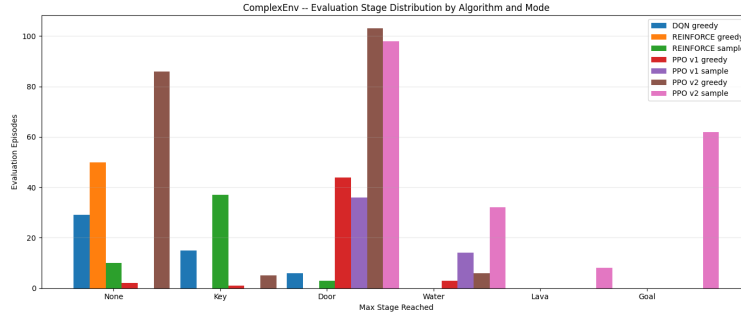


Fig. 16. Evaluation stage-distribution across all algorithm/mode combinations.

7. Discussion

7.1 MDP characterisation

Both environments are episodic, fully observable (the whole grid is rendered), and have discrete action spaces; from the agent's side, however, the observation is a high-dimensional image, and latent task state (what is carried, whether the door is open, which lava tiles remain) must be inferred from pixel colour alone, with no symbolic state exposed — the argument for why this is nonetheless Markov without frame stacking is given in Section 3.2. SimpleRoomEnv's underlying state space is small (8×8 cells $\times$ 4 facings $\times$ 4 goal corners $\approx 1,024$ configurations); a Q-table would be feasible in principle, but the 84×84 -pixel observation makes tabular RL impractical and motivates function approximation. ComplexEnv is vastly larger: 512 distinct layouts (16 key cells $\times$ 8 door rows $\times$ 2 goal corners $\times$ 2 water corners), each with up to 131,328 dynamic states (57 walkable cells $\times$ 4 facings $\times$ 3 carry states $\times$ 3 door states $\times$ 2^3 lava subsets $\times$ 2^3 water subsets) — a product upper bound of $\approx 6.7 \times 10^7$ configurations, for which deep function approximation is the only viable approach. Both environments deliver sparse reward (+1 on goal, 0 otherwise); within ComplexEnv, dropping the key or spending a water ball without extinguishing lava are recoverable mistakes, while stepping onto unextinguished lava is irrecoverable (episode terminates at reward 0) — this asymmetry is central to the shaping design and to the ablation in Section 8.

7.2 Headline results

ComplexEnv is solved. PPO with corrected reward shaping reaches the goal in **31.0%** of 200 held-out evaluation layouts (95% Wilson CI [**25.0%, 37.7%**]), from raw pixels, in ~ 1.79 M environment steps. DQN also completes the full five-stage chain, reaching the goal in 49 of 4,000 training episodes. On SimpleRoomEnv, DQN reaches 95.0% greedy success at a near-optimal mean path length of 14.1 steps. Three further findings, each backed by a controlled measurement rather than a reading of learning curves:

- A two-constant change to the ComplexEnv reward shaping took PPO from 0 successes in 4,000 episodes to 357 in 5,000 (Section 8) — global reward-budget safety and local risk-reward balance are independent constraints; satisfying only the first is not enough.
- Two algorithms reaching comparable SimpleRoomEnv scores did so by categorically different mechanisms: 88% of DQN's performance traces to state-conditioning, versus 93% of PPO's tracing to a state-independent action prior (Section 9).
- Deterministic (greedy) evaluation substantially understates both algorithm families: 5% action noise raises DQN's mean stage from 0.54 to 1.60; sampling instead of argmax raises PPO v2's mean stage from 1.15 to 3.17 and its success rate from 0% to 31% (Section 10).

7.3 Corrections applied after the first full run

The reported implementation was audited line-by-line against the assignment specification after an initial full run ("Run A"); five defects were found that materially affected the numbers, all fixed prior to the runs reported above.

#	Defect	Fix
1	PPO treated time-limit truncation as termination in the GAE mask, zeroing the bootstrap at nearly every ComplexEnv episode boundary and injecting a false terminal target on $\sim 100\%$ of episodes.	GAE now bootstraps $v(s, T)$ on truncation and only cuts the trace; true terminals still use $V=0$ (Section 2.3).

#	Defect	Fix
2	REINFORCE summed the policy loss over an episode but averaged the entropy term, so the effective entropy weight scaled as $\text{coef}/T \approx 10^{-5}$ - 10^{-4} of the policy gradient — entropy regularisation did nothing.	Both terms are now means over the episode's steps; default baseline switched to running statistics (measured against alternatives in Table 9).
3	PPO ran full-batch updates with no minibatching, and its KL early-stop fired after the optimiser step had already been applied.	Shuffled minibatches with per-minibatch advantage normalisation, orthogonal init, linear LR anneal, and the KL check moved before the step.
4	DQN's target-network sync was coupled to <code>train_freq</code> (checked inside <code>update()</code>), so the effective sync period was <code>lcm(target_update, train_freq)</code> rather than the configured value.	<code>maybe_sync_target()</code> now runs on the environment-step clock, called once per step; regression-tested to fire exactly 10 times in 100 steps at <code>target_update=10, train_freq=4</code> .
5	Shaping mass exceeded the discounted goal payoff (+13.5 of milestones vs. +1.34 discounted goal) — the shaped optimum plausibly excluded the goal, and Run A's PPO found exactly that optimum (farm milestones, then idle to the step cap).	Shaping rebalanced so discounted milestone mass is strictly below the discounted goal payoff (Section 1.3); the full story of what this got right and what it broke is Section 8.

Table 12. Defects found in Run A and their fixes.

Three further corrections affect reporting rather than learning: a chunk-merge bug had dropped `cumulative_steps` from the DQN/ComplexEnv logger, making DQN appear to have used ~5% of PPO's sample budget when it had used ~85%; model selection was asymmetric (DQN evaluated at its best checkpoint, the others at final weights) and is now one `BestCheckpointer` rule for all three algorithms; and training success had been measured under an $\epsilon=0.15$ behaviour policy against a greedy evaluation, resolved by the periodic greedy probes plotted alongside the behaviour-policy curve in every training figure above.

A sixth defect surfaced only at runtime, and only because of the memory-optimisation fix for defect 2: recomputing an entire episode's log-probabilities in one batched pass (rather than retaining a live autograd graph per step) first drove a convolutional forward pass to a batch of up to 400 three-channel observations. At that shape, on the T4 GPU used for these runs, cuDNN selected a convolution algorithm returning a non-contiguous tensor, and a reshape-by-view call raised a stride-incompatibility error; the fix was to use a copy-based flatten instead of a view. This class of defect — device- and shape-dependent — is not discoverable by static source review and depends on the algorithm a specific device selects for a specific input shape.

7.4 Independent verification of the applied corrections

Because an earlier draft of the discussion had asserted analyses and ablations that did not exist anywhere in the notebook, each fix above was re-verified at the source level rather than accepted from its own description: the GAE branch reproduces hand-computed advantages for a worked three-step rollout under both truncation and termination; the target-sync fires exactly ten times in one hundred steps at `train_freq=4`; the REINFORCE loss is invariant to episode length under the corrected mean reduction, where it previously scaled by $10\times$ between a ten-step and hundred-step episode; and the shaping-budget inequality holds for the current configuration and fails, as expected, for Run A's. A static audit confirms the code implements the intended algorithm, not that it attains any particular success rate — that claim is deferred entirely to the executed runs and their tables above — and it could not, by construction, have found the device-dependent cuDNN defect or questioned the chosen shaping magnitudes, both of which were only found by running the experiments.

8. Reward Shaping: A Controlled Ablation, and the Result That Decided the Project

Config	Milestone mass	Discounted ratio vs. goal	lava_bonus : death_penalty	Outcome
Run A	13.5	3.60 (fail)	3.0 : 2.0 (1.5:1)	Reward hacking — milestones, then idle
v1	3.45	0.56 (pass)	0.5 : 2.0 (1:4)	No hacking, but 0 successes / 4,000 episodes
v2	6.45	0.84 (pass)	1.5 : 1.0 (1.5:1)	357 successes / 5,000 episodes; 31% held-out

Table 13. Three shaping configurations, two failure modes, one that works. (PPO v2's budget was extended to 5,000 episodes, ~2M env steps, to allow convergence.)

Run A's failure is the textbook one (Section 1.3): the goal was worth only 1.34 discounted against 13.5 of milestone mass, a $3.60\times$ violation, and Run A's PPO converged to exactly the shaped optimum — key → door → water, then idle to the 400-step cap, sparse return 0.00, every episode.

v1 fixed that and introduced a different failure. With mass cut to 3.45 (ratio 0.56) the goal was safely optimal again, and PPO learned the first three stages well ($P(\text{key})=0.98$, $P(\text{door}|\text{key})=0.89$) before stopping dead: over 1.59M steps it extinguished lava exactly twice (both before episode 750) and never reached the goal. The funnel localises the break precisely at **P(extinguish | carrying water) = 0.004**.

The diagnosis: a global constraint satisfied, a local one violated. v1 paid +0.5 for extinguishing a lava tile and charged -2.0 for dying on one — a 1:4 bet against approaching lava at all, and the agent learned exactly that: 46 lava deaths, all before episode 2175, then zero for the remaining 1,825 episodes, while extinguish events had already ceased at episode 736. The shaping-budget audit checks a *global* quantity (total discounted milestone mass vs. the discounted goal) and v1 passes it comfortably; it does not check the *local* risk-reward ratio at the single decision that gates the rest of the chain. These are independent constraints:

A shaping scheme can be globally safe (the goal remains the optimal policy) and locally unlearnable (the action sequence leading to the goal is individually disincentivised at some step). The budget audit verifies the former; nothing in it verifies the latter.

The intervention. v2 changes two constants only — lava_bonus 0.5→1.5, death_penalty 2.0→1.0 — same seed, architecture, and optimiser settings, raising the local ratio from 1:4 against to 1.5:1 in favour while keeping the global audit satisfied (mass 6.45, ratio 0.84<1, pass). The predicted mechanism fired at every link in the chain:

Measure (training)	v1	v2	Change
Lava engagement (deaths + extinguishes)	48 eps (1.2%)	1,144 eps (22.9%)	19×
P(water door)	0.160	0.341	2.1×
P(extinguish water)	0.004	0.372	93×
P(goal extinguish)	—	0.625	—
Goal-reaching episodes	0 / 4,000	357 / 5,000	—
Max shaped return	−1.07	+24.46	—

Table 14. v1 vs. v2 mechanism confirmation. Late-run behaviour: SR=30.8% over episodes 4000–4500 and 28.2% over 4500–5000, best checkpoint at episode 4600 (mean stage 3.50); successful episodes average 152 steps, with the steps-per-successful-episode curve descending cleanly from 220 to ~110 — empty for every other run in this project. v2's printed milestones-then-idle return is +2.45 (positive, a nominal farming risk under Run A's logic) yet the agent reaches the goal in 152 steps rather than idling, confirming that the discounted inequality — not the undiscounted idle sum — is the criterion that governs whether the goal remains optimal.

Held-out evaluation (n=200, also Table 10): greedy SR 0.0% (mean stage 1.15, 54% reach Door, 0% reach Goal); sampled SR **31.0%** (mean stage 3.17, 100% reach Door, 51% reach Water, 35% reach Extinguish, 62/200 reach Goal). Against the random-policy floor measured below (Door 8%, Water 0%, Goal 0%), every stage transition is a large, unambiguous gain.

9. State-Blind Baselines: What Did Each Agent Actually Learn?

A success rate means little without knowing what the task gives away for free. Two control policies were measured on the same evaluation seeds as the agents: a uniform-random policy, and each agent's own marginal action distribution (preserving how often it takes each action while destroying any dependence on the observation) — decomposing performance into what action frequency buys and what conditioning on pixels buys.

9.1 SimpleRoomEnv: a strong trivial baseline

Agent	Uniform	Own marginal	Trained	Gain from marginal	Gain from conditioning
DQN	0.205	0.295	0.950	+0.090 (12%)	+0.655 (88%)
PPO	0.205	0.695	0.730	+0.490 (93%)	+0.035 (7%)
REINFORCE	0.205	0.525	0.515	+0.320 (103%)	−0.010 (−3%)

Table 15. DQN's conditioning gain is +0.655 ($z \approx 18$); PPO's +0.035 and REINFORCE's −0.010 are inside noise ($SE \approx 0.045$). DQN's competence is 88% state-conditioning; PPO's is 93% action prior — two agents at comparable scores via categorically different mechanisms. Episode length sharpens this: DQN's marginal takes 85.8 steps (indistinguishable from uniform's 87.2 — its action frequencies buy nothing) while DQN greedy takes 14.1, close to the ~7–8-step optimum; a state-blind policy that succeeds does so by wandering, and no action prior can produce a direct path.

Agent	Cross-state logit σ	Distinct argmax actions	Argmax distribution (L/R/Fwd)
DQN	0.39 – 0.45	{0, 1, 2}	33 / 19 / 48
REINFORCE	0.003 – 0.017	{2}	0 / 0 / 100
PPO	0.033 – 0.479	{2}	0 / 0 / 100

Table 16. Logit diagnostic, 100 distinct reset states per trained policy. Both policy-gradient agents return a constant action (forward) on every state, explaining why their greedy evaluations match to four significant figures across different seeds and baselines: their greedy policy walks straight until a wall, the ~5% that succeed already faced the goal, and the rest time out (mean 95.2 steps). The two cases differ in kind: REINFORCE's logits vary by at most 0.07 against a decision gap of 1.29 (functionally input-blind); PPO's vary substantially (action-2 logits span 1.67→3.41 — its CNN demonstrably reads the image) but the variation is confined to a single confidence direction that never reorders the actions — a state-dependent confidence in a fixed action, not a state-dependent choice. (DQN's lower cross-state σ here (0.39) versus an earlier undertrained DQN (2.19) reflects convergence, not less state-dependence: a converged Q-function assigns similar value to most reachable states while the action ordering within each state encodes the policy.)

9.2 ComplexEnv: where the task actually begins

Policy (n=50 unless noted)	Mean stage	≥Key	≥Door	≥Water	Goal
Random baseline	0.80	72%	8%	0%	0%
DQN greedy	0.54	42%	12%	0%	0%
REINFORCE greedy	0.00	0%	0%	0%	0%
REINFORCE sampled	0.86	80%	6%	0%	0%
PPO v1 greedy	1.96	96%	94%	6%	0%
PPO v1 sampled	2.28	100%	100%	28%	0%
PPO v2 sampled (n=200)	3.17	100%	100%	51%	31%

Table 17. A uniform-random policy scores mean stage 0.80, grabbing the key in nearly three-quarters of episodes — the Key stage is largely free, and 'P(key)=0.90' is a modest rather than impressive result. Door (random: 8%) is where learning becomes visible, and Water (random: 0%) is a genuine wall. REINFORCE never exceeded the baseline (sampled 0.86 vs. 0.80 is inside noise, and its training-time Door rate of 5.2% is below random's 8%; the early stop at episode 801 was the correct call). DQN's greedy policy is significantly worse than random (0.54 vs. 0.80, $z \approx -3.0$, $p \approx 0.002$) despite being the only algorithm besides PPO v2 to complete the full chain — resolved next. PPO v1 and v2 clearly beat the baseline, v1 at Door and v2 at every stage through the goal.

10. Determinism, Not Exploration: Why Greedy Evaluation Understates Both Families

DQN completes ComplexEnv's five-stage chain during training (49 goal-reaching episodes, full funnel P(key) 0.90 → P(door|key) 0.55 → P(water|door) 0.23 → P(extinguish|water) 0.24 → P(goal|extinguish) 0.44) yet scores mean stage 0.54 greedy — below random. An ϵ -sweep on the same 50 evaluation seeds resolves the apparent contradiction:

ϵ	Mean stage	$\geq \text{Water}$	$\geq \text{Extinguish}$	Goal	vs. random (0.80)
0.00	0.54	0	0	0	below
0.05	1.60	5	3	2	2.0×
0.15	1.76	10	5	0	2.2×

Table 18. The $\epsilon=0 \rightarrow 0.05$ jump is +1.06 mean stage ($z \approx 5.6$); the $0.05 \rightarrow 0.15$ step is +0.16 ($z \approx 0.7$, not significant) — performance saturates immediately. This is the finding: the binding constraint is determinism itself, not the quantity of exploration. DQN's Q-function holds competence the argmax policy cannot express because it enters loops, and ~5% stochasticity is the minimum dose needed to break them. PPO v2 shows the same phenomenon without any ϵ at all — greedy 0% / mean stage 1.15 vs. sampled 31% / 3.17 from the same weights; sampling acts as a retry mechanism, letting a policy that assigns 'toggle' 30% probability at the right state succeed within the step budget, where an argmax ranking it second never does. The penalty is larger for the better policy: v1 loses 0.32 mean stage going greedy, v2 loses 2.02, and v2 greedy (1.15) is actually worse than v1 greedy (1.96) despite v2 being decisively the stronger agent — v2's lower late-training entropy (~0.4–0.5 vs. v1's ~1.1) makes its argmax more committed and its greedy failure more bimodal (86/200 stuck at stage 0, 103/200 reaching Door).

	Mean stage
Training ($\epsilon=0.15$, last 100 eps at best checkpoint)	2.28
Held-out evaluation at $\epsilon=0.15$	1.76
Held-out evaluation at $\epsilon=0.00$	0.54

Table 19. Decomposing DQN's train/eval gap: determinism accounts for $1.76 - 0.54 = 1.22$ (70% of the gap); generalisation accounts for $2.28 - 1.76 = 0.52$ (30%). Matched on exploration, training and held-out layouts differ by only 0.52 stages — a modest, believable generalisation gap. Most of what looked like catastrophic train/eval divergence was deterministic policy extraction, not generalisation failure. A statistical footnote: DQN's training success rate is $49/4000 = 1.2\%$; at $n=50$ the probability of observing zero successes even if greedy matched the behaviour policy exactly is $0.988^{50} \approx 55\%$, and the Wilson 95% CI for 0/50 is [0%, 7.1%], which contains 1.2%. 0/50 at greedy is thus both a real degradation and an underpowered sample — which is why the PPO v2 headline was measured at $n=200$.

11. Hyperparameter Recalibration: A Hypothesis Proposed, Falsified, and Reverted

Fixing an estimator also invalidates hyperparameters implicitly tuned around its old, biased behaviour. Two rescalings were sustained and two exploration cuts were falsified by experiment and reverted — reported here because measuring the failures was the point.

- **Sustained — REINFORCE learning rate $5 \times 10^{-5} \rightarrow 3 \times 10^{-4}$.** Fixing the sum-vs-mean defect (Table 12, #2) shrank the raw gradient norm by a factor of order T, so the old rate would have understated the intended step by ~2 orders of magnitude. Outcome: sampled evaluation improved 46.0% (Run A) → 51.5%.
- **Sustained — PPO learning rate $1 \times 10^{-4} \rightarrow 2.5 \times 10^{-4}$.** With minibatching, orthogonal init, and correctly timed KL early-stop now in place, the more standard rate applies (Schulman et al., 2017; Huang et al., 2022). Outcome: SimpleRoomEnv sampled evaluation 56.0% → 73.0%, training SR-100 0.63 → 0.76.
- **Falsified — DQN/SimpleRoomEnv ϵ -decay 60,000 → 25,000 steps (with budget cut 2,000 → 1,200 episodes).** Greedy evaluation fell 90.5% → 53.5%, with rolling success peaking near episode 1,000 and then declining — the schedule was cut before the policy stabilised. **Reverted** to 60,000/2,000: 95.0% at 14.1 mean steps, exceeding Run A on both axes.
- **Falsified — DQN/ComplexEnv ϵ -floor 0.15 → 0.05 and train_freq 4 → 2.** Intended to align the training-time behaviour policy with greedy evaluation; instead caused catastrophic forgetting — DQN found the full chain around episodes 850–1,300 and then lost it entirely for the remaining 2,400 episodes. **Reverted** to 0.15/4: peak SR-100 rose 0.05 → 0.11, 49 goal-reaching episodes appeared across episodes 1113–3381 (rather than one vanishing window), and wall-clock fell 43% (15,388s → 8,837s) since halving the replay ratio halved the gradient steps.

Both failures share a lesson: exploration was load-bearing in ways the learning curves alone did not advertise, and greedy probes are an instrument for honest measurement, not a substitute for the exploration itself.

12. Algorithm Comparison

Property	DQN	REINFORCE	PPO
Family	Value-based (off-policy)	Policy-gradient (on-policy)	Actor-critic (on-policy)
Sample efficiency	High (experience replay)	Lowest (discards every transition)	Middle (rollout reused over epochs)
Stability	Target net + Huber loss	High variance (MC returns)	Clipped surrogate bounds updates
Exploration	ϵ -greedy + greedy probes	Stochastic policy + entropy bonus	Stochastic policy + entropy bonus
SimpleRoomEnv greedy	95.0%, 14.1 steps	5.0% (degenerate argmax)	5.0% (degenerate argmax)
SimpleRoomEnv sampled	n/a	51.5%	73.0%
State-conditioning gain	+0.655	-0.010	+0.035
ComplexEnv best held-out	mean stage 1.76 @ $\epsilon=0.15$	mean stage 0.86	31.0% success (v2)
ComplexEnv training successes	49 / 4,000	0 / 801	357 / 5,000 (v2)

Table 20. Cross-algorithm summary. PPO v2 budget extended to 5,000 episodes (~2M env steps) to allow convergence.

The two environments separate the algorithms differently. On SimpleRoomEnv only DQN learned to see: both policy-gradient methods settled into a state-blind forward bias, a comfortable local optimum since a forward-biased random walk already scores ~70% in an empty room and the sparse reward provides little pressure to do better; DQN, learning action values from replayed transitions rather than reinforcing whole trajectories, had no equivalent shortcut. On ComplexEnv the ordering reverses — PPO's milestone ladder can only be climbed by navigating to randomised key positions and door rows, so state-blind behaviour earns nothing and the gradient forces conditioning (PPO greedy reaches Door in 94% of episodes, impossible for a constant action under per-episode layout randomisation): dense structured shaping was not merely an accelerant here, on this evidence it was the precondition for the policy-gradient agents to use their observations at all. Off-policy discovery versus on-policy retention is the other axis: DQN's persistent 15% random floor kept it walking into the lava ring (492 deaths, 12.3% of episodes) but that same persistence let it discover toggle-while-facing-lava 111 times and complete the chain 49 times, whereas PPO v1's entropy bonus over a converged distribution learned pure avoidance instead (46 deaths, all before episode 2175) and never discovered the transition. Both greedy and sampled modes are reported for the policy-gradient methods throughout because their learned object is a distribution — argmax is a derived deterministic policy, not the policy itself, and the gap between them (Section 10) measures something real. All evaluation seeds occupy a space disjoint from training (`episode_seed(·, 'eval')`), and sampled-mode evaluation reseeds the torch RNG per episode for reproducibility.

13. Key Design Choices

- **Artifact checkpointing.** All logger metrics, figures, and model weights (final and best-so-far) are flushed to disk as training proceeds, with the logger autosaved every 250 episodes alongside a checkpoint carrying current weights, step counter, and episode index — the resilience patch that makes a multi-hour Colab run survivable across free-tier disconnects.
- **Reload from checkpoints.** A single flag rebuilds every agent and logger from saved artifacts, so the notebook runs top-to-bottom in minutes with no retraining, and every figure and table above regenerates from the exact stored state used for these numbers.
- **One source of truth for hyperparameters.** Tables 4–6 are consumed directly by the training calls rather than re-typed, so the settings stated here are necessarily the settings that ran; each revert in Section 11 was a one-line change in one place.
- **Regression tests, not just sanity checks.** Every automated assertion targets a defect that was actually present: hand-computed GAE targets for truncation vs. termination, target-sync frequency under `train_freq=4`, episode-length invariance of the REINFORCE loss, replay-buffer next-state linkage, and legacy-log repair.
- **A documented early-stopping rule.** REINFORCE/ComplexEnv stops when neither rolling success rate nor rolling mean stage improves by 0.01 over 800 episodes; it fired at episode 801, converting an otherwise undefended short run into a measured budget decision and freeing ~75 minutes of GPU time that went into the v2 ablation instead.
- **Shaping reads event counts only** (carrying flags, door-open flag, extinguished-lava count, `n_lava` from info) — no position getter is consulted inside the reward path, so no map geometry can leak into the reward.

14. What the Shaping Bought, and What It Cost

- A first-pickup-only water bonus removes the pickup/drop farming exploit that had inflated an earlier run's shaped returns.
- The key bonus is the milestone a replay buffer amplifies most: DQN reaches the Key stage in 90% of training episodes.
- The lava-death penalty controls the approach/avoid balance, and (Section 8) it is its *ratio* to the lava bonus, not its absolute size, that determines whether stages 4–5 are reachable at all.

- What shaping cannot do: manufacture a gradient toward an event that never occurs. For REINFORCE, milestone events past the key were rare enough that the shaped return was dominated by the step penalty alone ($-0.01 \times 400 = -4.0$), carrying no directional signal — no reward design fixes an event rate of zero.
- The step penalty discourages wasted motion but also penalises the ~ 100 – 150 steps of backtracking stage 1 genuinely requires; it is retained because the shaping-budget argument needs it large enough to make idling unprofitable, a real tension rather than a free choice.
- Sparse and shaped returns are tracked independently at every stage (the sparse tracker sits innermost, before any shaping wrapper touches the reward) — this separation is what made the v1 diagnosis possible: an agent accumulating shaped reward at sparse return 0.00 looks like it is learning until the two are plotted apart.

Potential-based shaping (Ng, Harada & Russell, 1999), $F(s, s') = \gamma \Phi(s') - \Phi(s)$, is policy-invariant for any bounded Φ and would remove the magnitude-tuning problem entirely — the “strong enough to survive fading exploration” versus “weak enough not to be farmed” trade-off would stop being a trade-off. This project's ablation is a direct empirical motivation for that approach and is the extension we would implement next.

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

15. Why PPO Over A2C

The assignment requires at least one actor-critic method; PPO was implemented directly rather than starting from A2C, since PPO on SimpleRoomEnv already validates the shared CNN backbone, actor-critic heads, GAE computation, and rollout loop equally well; the incremental complexity over A2C (the clipped surrogate and multiple epochs per rollout) is small; and PPO's clip bounds the per-update policy change in a way A2C's unconstrained update does not, which mattered on ComplexEnv's long-horizon setting. The choice is vindicated by the outcome — PPO is the algorithm that ultimately solved ComplexEnv — though it is worth noting that Run A's early PPO struggles (truncation bootstrapping, minibatching, KL timing) were implementation defects rather than an algorithm-choice problem, a caution against attributing to an algorithm what belongs to its implementation.

16. Limitations

- **Single seed per configuration.** Every result is one run; deep RL is notoriously seed-sensitive. The controlled comparisons (v1 vs. v2 shaping, the ϵ -sweep, the state-blind baselines) share seeds and are therefore paired, which strengthens them considerably, but a multi-seed protocol with IQM and stratified bootstrap CIs (Agarwal et al., 2021) is what this analysis would need to be publication-grade; the free-tier compute budget did not permit it.
- **The v2 shaping was one intervention, not a sweep.** `lava_bonus=1.5`, `death_penalty=1.0` was chosen to invert the local ratio while satisfying the global audit; it was not optimised. The result establishes that the local ratio was the binding constraint, not that these are the best possible values.
- **PPO v2's headline evaluation is sampled, not greedy** (its greedy score is 0%); we report and justify both (Section 10), but a reader who insists on deterministic inference should read the greedy row of Table 10.
- **The logit diagnostic samples reset states**, not the full state-occupancy distribution of a rollout; the eval statistics (95.2-step means, i.e. never turning from a wall) independently confirm the constancy extends along trajectories, but the sample is not the exact marginal.
- **ComplexEnv is not solved reliably.** 31% is a solve rate, not a solution; the remaining 69% are dominated by episodes that reach Water and stall — the extinguish chain still fails more often than it succeeds.

17. Conclusion

The project's decisive result is not any single number but the chain of reasoning that produced it. ComplexEnv appeared unsolvable across three algorithms and roughly 3.5M environment steps. Measuring the stage funnel rather than the success rate localised the failure to one transition, $P(\text{extinguish} \mid \text{carrying water}) = 0.004$; inspecting the reward at that transition revealed a 1:4 incentive against the required action, invisible to a shaping audit that checks only global budget. Changing two constants to invert that ratio, while keeping the global audit satisfied, raised the transition rate 93-fold and produced 357 goal completions and a 31% held-out success rate, with the predicted mechanism — increased lava engagement, then increased extinguish rate, then goal completion — confirmed at every link.

The supporting analyses matter as much as the headline. State-blind baselines showed that SimpleRoomEnv hands 69.5% success to a fixed forward bias (via the marginal-baseline decomposition) and ComplexEnv hands 72% of the Key stage to a random policy, reframing several results that would otherwise have looked stronger than they were. The competence decomposition showed DQN and PPO reaching comparable SimpleRoomEnv scores by opposite mechanisms — 88% state-conditioning versus 93% action prior. The determinism analysis

showed greedy evaluation understating both algorithm families substantially, splitting DQN's train/eval gap into 70% determinism and only 30% generalisation.

Two of our own hyperparameter recommendations were falsified by measurement and reverted; both reverts then improved results on independent environments, one of them also cutting wall-clock by 43%. Reporting that cycle — hypothesis, measurement, falsification, revert, confirmation — rather than only the configuration that finally worked, is the part of this project we would defend most strongly: the same standard applied to the initial implementation's defects was applied to our own corrections, our own hyperparameters, and the limits of the static audit that started it. What holds throughout is the methodological core — sparse and shaped returns reported separately at every stage, evaluation on a disjoint seed space with greedy and sampled modes both shown, one model-selection rule for every algorithm, all runs on a common environment-step axis, stage progress reported as a distribution rather than a best case, and every headline rate accompanied by the baseline it should be read against. Those are the parts that make the comparisons mean something, and they are also what made every defect above findable.

References

- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- Engstrom, L. et al. (2020). Implementation Matters in Deep RL: A Case Study on PPO and TRPO. *ICLR 2020*.
- Huang, S. et al. (2022). The 37 Implementation Details of Proximal Policy Optimization. *ICLR Blog Track*.
- Pardo, F., Tavakoli, A., Levdik, V., & Kormushev, P. (2018). Time Limits in Reinforcement Learning. *ICML 2018*.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *ICML 1999*.
- Agarwal, R. et al. (2021). Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS 2021*.